# Understanding ML: The Experiments

**Train / validation / test, cross-validation, learning curves, bias–variance experiments, hyperparameter tuning, and why the test set must not be touched: what is manipulated, what is measured, what the curves mean, what people do wrong**

Fundamentals of Machine Learning, midterm

2026-11-03

## What this document is about, in one paragraph

Every method of Lectures 1–5 is judged by the same experiment: fit on some data, measure on other data, and vary one thing at a time. The thing we vary is called the *manipulated variable*: the amount of data $N$, the capacity (degree, depth, $k$, $\lambda$), the number of training epochs, the number of trees, the split protocol. The thing we measure is always a *risk* (an average loss) on a set of examples, and the whole art is choosing *which* examples, because the examples that chose the model cannot also grade it. This document describes each of the standard experiments as a procedure you could run tomorrow, says what the plot looks like, how to read each shape, and what people do wrong. Formulas marked [on your sheet] are on your two-page sheet; do not memorise them, understand what they measure.

## Contents


**1 The one experiment behind everything** **2**

**2 Train, validation, test: the split** **3**

**3 Why the test set must not be touched** **6**

**4 Cross-validation** **8**

**5 Hyperparameter tuning** **10**

**6 Learning curves: vary the amount of data** **13**

**7 Bias–variance experiments** **15**

**8 The complete protocol, start to finish** **17**

# 1 The one experiment behind everything

## Risk on a set of examples

A model $f$ and a loss $\ell$ give, on any set of examples $S$, the *empirical risk* $\widehat{R}_S(f) = \frac{1}{|S|}\sum_{i\in S}\ell(y^{(i)}, f(x^{(i)}))$ [on your sheet] and, on the whole population, the *population risk* $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet]. The population risk is what we want (performance on the next case) and can never be computed; every experiment is a way of estimating it with an empirical risk on examples the model has not used.

**Purpose.** To replace the unknowable number $R(f)$ by a number we can compute, without fooling ourselves.

**Impact.** The choice of the set $S$ decides whether the number is honest. On examples the model never saw, $\widehat{R}_S(f)$ is an unbiased estimate of $R(f)$; on the examples that chose $f$, it is optimistic (Section 2).

## The repeated-sampling thought experiment

Imagine drawing a training set $\mathcal{D}$ from the world, running the whole learning procedure, and getting a fitted model $\hat{f}_{\mathcal{D}}$. Now imagine doing it again with a fresh $\mathcal{D}'$: you get a different $\hat{f}_{\mathcal{D}'}$. Every quantity in this course that has a hat is a random variable through $\mathcal{D}$. "Bias", "variance", "optimistic", "unbiased" are all statements about what happens *on average over such repetitions.*

**Purpose.** To make "sensitive to the sample" and "honest estimate" precise: they are averages $\mathbb{E}_{\mathcal{D}}[\cdot]$ over training sets.

**Impact.** Any experiment you run once gives one draw. Repeating it with different seeds, folds or bootstrap samples is how you see the variability that the theory talks about.

## The generic learning experiment

**Setup.** A dataset, a learning procedure with one knob $h$ (degree, $k$, $\lambda$, depth, epochs, number of trees, or the amount of data $N$), a loss, a metric.

**Held fixed.** Everything except the knob: the same rows in each subset, the same preprocessing, the same seed where randomness enters, the same metric.

**What we manipulate.** The knob $h$, one value at a time, over a grid.

**What we measure.** Two numbers per value of $h$: the risk on the examples used for fitting (*training* risk) and the risk on examples not used for fitting (*held-out* risk).

**What the plot shows.** Two curves against $h$. The training curve, and the held-out curve above it. Where the two sit, how far apart they are, and whether they still move, is the diagnosis.

**How to read it.** Three questions in order. (1) Are both curves high? The procedure is too restricted or the representation is poor (underfitting). (2) Is the gap between them large? The fit depends on the sample (overfitting, high variance). (3) Are the curves still moving as the knob increases? You have not reached the regime where the knob stops mattering.

**The four curves you will be shown, and what is on the horizontal axis**

| Curve | Manipulated variable | Typical shape | What it diagnoses |
|---|---|---|---|
| Validation (complexity) curve | capacity: degree, depth, $1/k$, $1/\lambda$ | training falls monotonically; held-out falls then rises (U) | the best capacity; underfitting on the left, overfitting on the right |
| Training curve | epochs or boosting rounds | training falls; held-out falls then rises | when to stop (early stopping); same U over time instead of over size |
| Learning curve | amount of training data $N$ | training rises toward a plateau; held-out falls toward the same plateau | bias (plateau height) versus variance (gap) versus data-limited (still converging) |
| Ensemble-size curve | number of models $M$ | held-out falls then plateaus | when more models stop helping; the floor $\rho\sigma^2$ |

Mixing these up is the most common reading error: "the error goes down with more" means nothing until you say more *what*.

**Key insight.** Every experiment has the same skeleton: hold everything fixed, vary one knob, measure the risk twice (fitting examples versus unseen examples), and read the two curves. The method you are testing only decides what the knob is.

**How the professor asks it**

**Q.** *A colleague reports that "the error decreases when we increase the model". Explain why this sentence is not yet a result, and what two numbers you would ask for.*
**A.** It is not a result because it does not say which error. Training error *always* decreases (weakly) when capacity increases, because a larger nested family contains the smaller one's best fit [on your sheet]; so a falling training error carries no information about new cases. Ask for the held-out error at each capacity, and for the gap between the two. If the held-out error also falls, capacity was the limiting factor (underfitting); if it rises while the training error falls, the extra capacity is fitting the sample (overfitting).

## 2 Train, validation, test: the split

**The three subsets and their roles**

Split the available examples into three disjoint sets. The *training set* fits parameters. The *validation set* chooses between alternatives (hyperparameters, features, model families, thresholds, stopping epoch). The *test set* is used once, at the end, to report the performance of the single chosen model. [on your sheet]

**Purpose.** Each subset answers one question, and each question would be answered dishonestly by the examples used for the previous one.

**Impact.** Training examples give an optimistic number for the fitted model; validation examples give an honest number for *each* candidate but an optimistic number for the *winner*; only the test set gives an honest number for the winner, and only once.

**Why the training set grades too kindly, and the validation set does not**

Let $f$ be *any fixed* model and $S$ a set of $n$ i.i.d. examples. Then $\widehat{R}_S(f) = \frac{1}{n}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ is an average of $n$ i.i.d. copies of the random variable $\ell(Y, f(X))$, so $\mathbb{E}[\widehat{R}_S(f)] = R(f)$: honest. [on your sheet]

Now let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over a family $\mathcal{H}$, and let $f^*_{\mathcal{H}}$ minimise $R$ over $\mathcal{H}$. For every sample, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ because $\hat{f}_{\mathcal{D}}$ is the minimiser. Take expectations: the right side is $R(f^*_{\mathcal{H}})$ by the honest-estimate fact (since $f^*_{\mathcal{H}}$ is fixed). And $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ pointwise because $f^*_{\mathcal{H}}$ is the best in the family. So

$$\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]. \quad \text{[on your sheet]}$$

The training score of the fitted model sits *below* the best achievable risk, and the true risk of the fitted model sits *above* it: the training number is optimistic by at least the whole gap. The one word that made the difference is *fixed*: the validation set grades a model that was fixed before the validation examples were looked at, so for it the first line applies.

## Honest versus optimistic, seen directly

**Setup.** A simulated world where you know the truth (e.g. $y = \sin(2\pi x) + \varepsilon$), a model family (polynomials of degree 5), and a large pool of fresh test points.

**Held fixed.** The family, the noise level, the test pool.

**What we manipulate.** Nothing but the random training sample: repeat 200 times, each time drawing a fresh training set of $N = 20$ points.

**What we measure.** For each repetition, (a) the training risk of the fitted polynomial on its own 20 points, (b) its risk on the fresh test pool, and (c) the training risk of one *fixed* polynomial chosen before any data were drawn.

**What the plot shows.** Three histograms. (c) is centred on that fixed model's true risk. (a) is centred well below the centre of (b).

**How to read it.** The distance between the centres of (a) and (b) is the expected generalisation gap $\mathbb{E}[G]$ [on your sheet]; it is positive, and it grows with capacity and shrinks with $N$. The histogram (c) shows that the problem is not "training data", it is "the data that chose the model".

## The split protocol, as a procedure

**Setup.** $N$ labelled rows and a deployment question ("will this flow next month be an attack?", "does the model work on a new hospital?").

**Held fixed.** The rows in each subset, written down *before* any model is fitted, and never moved.

**What we manipulate.** The split rule, chosen from the deployment question: random when new cases are independent draws from the same population; *stratified* random to keep class proportions in each subset; *grouped* when deployment meets new users, devices, patients or sites (every group entirely in one subset); *temporal* when the model predicts the future (train on the past, validate on later, test on the latest). [on your sheet]

**What we measure.** Metrics on validation during development; metrics on test once.

**What the plot shows.** Not a curve: a table, one row per protocol, same model. The random-split number is the largest, the grouped number smaller, the temporal number smallest.

**How to read it.** The protocol that mirrors deployment gives the only number that predicts deployment. The others are not wrong arithmetic; they answer a question nobody asked.

## Intrusion detection: the same dataset, three verdicts

Flows recorded from 50 devices over 6 months. Random split: flows from every device and week are in both train and test; the model can learn "device 17 is the compromised laptop" and scores 99.5%. Grouped split (hold out 10 devices): the model must recognise attack behaviour, 96%.

Temporal split (train months 1–4, test months 5–6, during which a new attack tool appeared): 90%. Only the last number describes next month. A bank predicting churn "next quarter" is in the same situation: features must come from before the quarter and the outcome from after it, so the split is temporal.

## Choosing the split sizes

With 10,000 rows a common split is 60/20/20. The validation set must be large enough that the differences you want to detect are larger than its noise: the standard error of an accuracy estimate near 0.9 is $\sqrt{0.9 \cdot 0.1/n}$, which is 0.03 for $n = 100$, 0.0095 for $n = 1{,}000$ and 0.003 for $n = 10{,}000$. Comparing two models that differ by half a percent on 100 validation rows is reading noise. With few rows, use cross-validation (Section 4) instead of a single small validation set.

## The leaks: using tomorrow's information today

Leakage is any use, during development, of information that will not exist when a new decision is made [on your sheet]. Each form inflates the held-out number, sometimes to perfection.

- **Preprocessing fitted on all data.** Standardising, imputing, selecting features or reducing dimension with means, variances and scores computed on train+validation+test. Fix: fit every preprocessing step on the training rows only and reuse the fitted $\hat{\mu}_j, \hat{\sigma}_j$ unchanged on validation, test and deployment [on your sheet].
- **Duplicates and near-duplicates** in several subsets; **the same device, session or patient** in train and test. Fix: deduplicate, then group.
- **Features created after the outcome**: "ticket closed as intrusion", "number of analyst comments". They predict perfectly and are useless at decision time. Fix: for each feature ask "at what time does this value become known?".
- **Shuffling a time series** before splitting: the model sees the future of the series it is tested on.
- **Hyperparameters chosen on test results** (Section 3).
- **Stratifying across groups**: scattering one device's flows across subsets to match class proportions destroys the grouping.

## Training set versus validation set versus test set

| | Training | Validation | Test |
|---|---|---|---|
| Used for | fitting parameters $\theta$ | choosing $h$: $\lambda$, $k$, depth, features, threshold, epoch | reporting the final number |
| How often looked at | every iteration | every candidate | once |
| Honest for | nothing after fitting | each candidate, not the winner | the winner |
| Becomes dishonest when | always | you pick the minimum over many candidates (winner's curse) | you use its result to change anything |

**Key insight.** The data that choose a model cannot grade it. The training set chooses $\theta$, the validation set chooses $h$, so each of them grades its own choice too kindly; the test set chooses nothing, which is the only reason its number can be believed.

**How the professor asks it**

**Q.** *You standardise all 10,000 rows, then split 80/20, train logistic regression and report 94% on the 20%. Is the 94% trustworthy? Explain exactly what leaked and how large you expect the effect to be.*
**A.** Not fully. The means and standard deviations used to scale the training rows were computed with the test rows included, so the test rows influenced the representation they are then graded on. For standardisation on 10,000 rows the effect is usually small, because two sample means differ little; but the same mistake with feature selection (choosing the 20 most correlated features on all rows) or with imputation by a fitted model can be large, and in cross-validation with few rows it is routinely several points. The fix costs nothing: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

**Q.** *Explain why a model that scores 99.5% on a random split of device-level intrusion data may score 90% in deployment, and name the split that would have predicted the 90%.*
**A.** The random split places flows of the same device, and the same weeks, in both subsets, so the learner can exploit device identity and period-specific patterns that recur in the test rows: a shortcut that holds inside the collection and not outside it. In deployment the model meets new devices and later weeks, where the shortcut is absent. A temporal split (and, for new devices, a grouped split) removes the shortcut from the evaluation and would have shown the lower number before deployment.

## 3 Why the test set must not be touched

**The winner's curse**

Evaluate many candidates on the same validation set and pick the one with the best score. The best score is the *minimum of several noisy estimates*, and a minimum of noisy numbers is pulled below the true value of whatever it picked. The reported validation score of the winner is therefore optimistic, even though each candidate's score was honest.

**Purpose.** To explain why a third set is needed at all, and why that set can be used only once.

**Impact.** The more candidates you compare, and the noisier the validation estimate, the larger the optimism. Using the test set to choose anything turns it into a validation set, and the optimism returns.

**The minimum of honest estimates is optimistic**

Let $\widehat{R}_{\text{val}}(\hat{f}_h)$, $h \in \mathcal{C}$, be validation risks of models fitted without the validation rows, so each is honest: $\mathbb{E}[\widehat{R}_{\text{val}}(\hat{f}_h)] = R(\hat{f}_h)$. For any particular $h_0$, the minimum over all $h$ is at most the value at $h_0$: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ for every sample. Take expectations: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le R(\hat{f}_{h_0})$. This holds for every $h_0$, so it holds for the best one:

$$\mathbb{E}\Big[\min_{h\in\mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)\Big] \;\le\; \min_{h\in\mathcal{C}} R(\hat{f}_h). \quad \text{[on your sheet]}$$

The inequality is strict as soon as the estimates are noisy and no single $h$ wins in every sample. How large? For two candidates with equal true risk $R$ and independent Gaussian estimation noise of standard deviation $\sigma$, $\mathbb{E}[\min] = R - \sigma/\sqrt{\pi} \approx R - 0.56\sigma$ [on your sheet]; for $n$ candidates the optimism grows like $\sigma\sqrt{2\ln n}$ [on your sheet]: $1.18\sigma$ for $n = 2$, $2.15\sigma$ for $n = 10$, $3.0\sigma$ for $n = 100$, $3.7\sigma$ for $n = 1{,}000$. The dependence on $n$ is slow (logarithmic), the dependence on the noise $\sigma$ is linear: a small validation set hurts more than a long list of candidates.

### Adaptive overfitting of a validation set

**Setup.** Labels that are pure coin flips (no signal at all), 1,000 rows, split 500/500.

**Held fixed.** The data; every "model" is a random rule that ignores $x$.

**What we manipulate.** The number $n$ of random rules tried: 1, 10, 100, 1,000, 10,000. For each, keep the rule with the best validation accuracy.

**What we measure.** The best validation accuracy, and the accuracy of that same rule on the 500 test rows.

**What the plot shows.** Best validation accuracy rises with $n$: about 50%, 53%, 56%, 57%, 59%. Test accuracy of the winner stays at 50% for every $n$.

**How to read it.** Nothing was learned, yet the validation number "improved" steadily, because you selected the luckiest of $n$ coin-flip scores. Any real development loop does a weaker version of this: every look at the validation score steers the next choice. The flat test line is what an untouched test set buys. If you now start choosing *among test scores*, the test line starts rising the same way and you have no honest number left.

### Public leaderboards

In data-science competitions the public leaderboard is scored on a fixed subset. Teams that submit hundreds of times and keep whatever scores best on it routinely drop many places on the private leaderboard, which is scored on rows they never saw. The public set has become their validation set; the private set is the untouched test set. The same happens inside a company when "the test set" is re-run after every change: the reported number drifts upward while deployment performance does not.

### How the test set gets touched without anyone noticing

- Running the test set "just to see" after each change, then keeping the change if the test number improved. That is selection on the test set.
- Choosing the decision threshold, or the metric to report, by looking at test results.
- Tuning on cross-validation over all rows, then reporting the cross-validation score as if it were a test score. The folds chose the hyperparameters; they are a validation estimate of the winner, hence optimistic.
- Early stopping on the test loss. The stopping epoch is a hyperparameter; it must be chosen on validation.
- "Fresh" test sets that share devices, sessions or time with the training rows.

Fix: freeze the test rows before development; use them once; if you must continue developing after seeing the test result, obtain new test data or use nested cross-validation (Section 4) and report the outer estimate.

**Key insight.** A validation score is honest for each candidate and optimistic for the winner; the optimism grows linearly with the noise of the estimate and only logarithmically with the number of candidates. The test set is the one set that chose nothing, so touch it once.

### How the professor asks it

**Q.** *You tried 50 hyperparameter configurations on a validation set of 200 rows and the best scored 91%. Why should you expect less than 91% in deployment, and what two things would reduce the discrepancy?*

**A.** The 91% is the minimum validation error among 50 noisy estimates, and the expected

minimum of honest estimates lies below the best true risk; with 200 rows the estimation noise is about $\sqrt{0.09 \cdot 0.91/200} \approx 0.02$, and the optimism for 50 candidates is of the order of $\sigma\sqrt{2\ln 50} \approx 2.8\sigma \approx 0.06$, so a few points of optimism are plausible. Reduce it by (1) a larger validation set or cross-validation, which shrinks $\sigma$ (the linear factor), and (2) a coarser or smaller search, which shrinks $\sqrt{2\ln n}$ (the logarithmic factor). To *know* the deployment number, evaluate the winner once on a test set that took no part in the 50 comparisons.

# 4 Cross-validation

### $K$-fold cross-validation

Partition the development rows into $K$ folds of equal size. For each fold $j$, fit the procedure on the other $K-1$ folds and evaluate it on fold $j$; average the $K$ scores:

$$\widehat{R}_{\mathrm{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}_{\mathrm{val}}^{(j)}(\hat{f}_h^{(-j)}). \quad \text{[on your sheet]}$$

Every row is validated exactly once and used for fitting $K-1$ times.

**Purpose.** To estimate the held-out risk of a *procedure* (with configuration $h$) when the data are too few to spare a large fixed validation set, and to compare configurations on all the data instead of one slice.

**Impact.** Replaces one noisy validation number by the average of $K$ numbers computed on $K$ different held-out slices, at the cost of $K$ fits per configuration. What it estimates is the risk of "run this procedure on $N(K-1)/K$ rows", not the risk of any one of the $K$ fitted models.

### The cross-validation protocol

**Setup.** Development rows (everything except the frozen test set), a candidate set $\mathcal{C}$ of configurations, $K$ (5 or 10 are standard).

**Held fixed.** The fold assignment, drawn once and reused for every configuration, so that all candidates are judged on the same slices. Grouped or temporal folds if deployment demands it.

**What we manipulate.** Which fold is held out (inner loop over $j$); which configuration is run (outer loop over $h$).

**What we measure.** $\widehat{R}_{\mathrm{val}}^{(j)}$ for each $(h, j)$; their mean $\widehat{R}_{\mathrm{CV}}(h)$ and their spread across folds.

**What the plot shows.** $\widehat{R}_{\mathrm{CV}}(h)$ against $h$, with error bars from the fold-to-fold spread. The curve is the validation curve of Section 5, now with a measure of its noise.

**How to read it.** Pick $\hat{h}$ at the minimum, but treat configurations whose curves lie within the error bars as tied; among ties prefer the simpler (the "one standard error" habit). Then *refit the chosen configuration on all development rows*; the $K$ fold models were only instruments. The score at $\hat{h}$ is still a validation score of a winner, hence optimistic; the test set gives the honest number.

### What the cross-validation estimate is, and its cost

Fold $j$ holds $N/K$ rows that the model $\hat{f}_h^{(-j)}$ never saw, so $\widehat{R}_{\mathrm{val}}^{(j)}(\hat{f}_h^{(-j)})$ is an honest estimate of $R(\hat{f}_h^{(-j)})$, the risk of a model trained on $N(K-1)/K$ rows. Averaging the $K$ of them estimates the *expected* risk of the procedure at that training size. With $K = N$ (leave-one-out) the training sets are almost the full data, so the estimate is almost unbiased for the full-data procedure, but the $N$ fitted models are nearly identical, so the $N$ scores are highly correlated and their average has high variance, and the cost is $N$ fits. With $K = 5$ or 10 the training sets are 80–90% of the

data (slightly pessimistic, since the final model sees more), the folds are more different (lower variance of the average), and the cost is 5 or 10 fits per configuration: 20 configurations at $K = 5$ is 100 fits.

**Spam filter with 600 labelled emails**

A single 60/20/20 split leaves 120 validation emails: an accuracy difference of 2% is below its noise. Five-fold cross-validation on the 480 development emails validates every email once and gives five numbers per configuration; their average is far less noisy and their spread tells you whether "$\lambda = 0.1$ beats $\lambda = 1$" is real. After choosing $\lambda$, refit on all 480, then score the 120 frozen test emails once.

**Cross-validation done wrong**

- **Preprocessing or feature selection outside the folds.** Selecting the top features on all development rows, then cross-validating the classifier, lets every validation fold influence the representation it is scored on. With many features and few rows this alone can produce near-perfect cross-validation scores on random labels. Fix: the entire pipeline (scaler, imputer, selector, model) is refitted inside each fold on its training portion [on your sheet].
- **Random folds on grouped or temporal data.** Rows of the same patient, device or session land on both sides; a time series is shuffled. Fix: grouped folds (each group in one fold) or forward-chaining folds (train on the past, validate on the next block).
- **Reporting the cross-validation score of the winner as the final estimate.** It chose $\hat{h}$, so it is a validation score. Fix: a test set, or *nested* cross-validation: an outer loop whose held-out folds are never used for selection, and an inner cross-validation, inside each outer training portion, that chooses $h$. The outer average is honest for the whole procedure "select then fit".
- **Deploying one of the $K$ fold models**, or averaging them, instead of refitting on all development rows. (Averaging them is a bagging-like ensemble, legitimate but a different method.)
- **Different fold assignments for different candidates.** Then the differences between candidates include the differences between slices.

**Validation split versus cross-validation versus out-of-bag versus nested cross-validation**

| | What is held out | Cost | Honest for |
|---|---|---|---|
| Single validation set | one fixed slice | 1 fit per candidate | each candidate; noisy if small |
| $K$-fold CV | each fold in turn | $K$ fits per candidate | each candidate; less noisy; still optimistic for the winner |
| Out-of-bag (bagging) | the $\approx 36.8\%$ rows missing from each bootstrap sample | free (a by-product of bagging) | the fixed ensemble; same winner's-curse caveat if used to select |
| Nested CV | outer folds for estimation, inner folds for selection | $K_{\text{out}} \times K_{\text{in}}$ fits per candidate | the whole select-then-fit procedure |

**Key insight.** Cross-validation changes which rows are held out, not the rule that held-out rows must have played no part in anything that was fitted or chosen. Everything data-dependent goes inside the fold.

**How the professor asks it**

**Q.** *With 300 rows and 5,000 features, a student ranks the features by correlation with the label on all 300 rows, keeps the best 20, and then runs 10-fold cross-validation of logistic regression on those 20, obtaining 97%. The labels are in fact unrelated to the features. Explain the 97%.*
**A.** With 5,000 features and 300 rows, by chance alone some features will be strongly correlated with the labels on this sample; ranking on all rows picks exactly those lucky features. Every cross-validation fold then validates on rows that already took part in choosing the features, so the "held-out" rows are not held out from the selection step: leakage of the validation labels into the representation. Running the selection inside each fold, on the 270 training rows only, would pick different lucky features each time and the validation folds would score near 50%.

**Q.** *Why does leave-one-out cross-validation have high variance although each of its $N$ fitted models uses almost all the data?*
**A.** Its $N$ training sets differ by one row, so the $N$ fitted models are nearly identical and their $N$ held-out losses are strongly correlated; the average of correlated quantities does not shrink its variance like $1/N$ (the same mechanism as the ensemble variance formula $\sigma^2(\rho + (1-\rho)/M)$ [on your sheet], with $\rho$ close to one). Five- or ten-fold folds are more different from each other, so their average is more stable, at the price of a slight pessimism because each model sees 80–90% of the rows.

# 5 Hyperparameter tuning

**Parameters, hyperparameters, and the selection loop**

A *parameter* $\theta$ is set by the training algorithm from the training rows (weights, coefficients, split thresholds, leaf values). A *hyperparameter* $h$ is a choice the training algorithm cannot make for itself because training error would always push it to maximal capacity: $\lambda$, $k$, degree, depth, number of trees, learning rate, number of epochs, $m_{\text{try}}$, the feature subset, the decision threshold. The selection loop is

$$\hat{\theta}_h \in \arg\min_{\theta} \widehat{R}_{\text{train}}(\theta; h), \qquad \hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h), \quad \text{[on your sheet]}$$

inner problem on training rows, outer problem on validation rows (or cross-validation).

**Purpose.** To choose the capacity, the regularisation and the stopping point from data without letting the training objective decide, since the training objective is biased toward flexibility.

**Impact.** Each hyperparameter moves the model along the validation curve: too little capacity on the left (both errors high), too much on the right (training low, validation high).

**Why hyperparameters cannot be chosen on the training set**

Regularised empirical risk minimisation $\hat{\theta}_\lambda \in \arg\min_\theta [\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)]$ [on your sheet] is equivalent to minimising $\widehat{R}_{\text{train}}$ subject to $\Omega(\theta) \le c$ with $c = \Omega(\hat{\theta}_\lambda)$ [on your sheet]; a larger $\lambda$ gives a smaller $c$ and a smaller family $\mathcal{H}_c$. The families are nested, $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, so the minimum training risk over $\mathcal{H}_{c_2}$ is at most that over $\mathcal{H}_{c_1}$ [on your sheet]. Hence training risk is minimised at $\lambda = 0$: choosing $\lambda$ on training data always answers "no regularisation". The same argument with degree, depth, $1/k$ or the number of epochs shows that every capacity hyperparameter is pushed to its maximum by training error. That is the whole reason validation exists.

## The validation curve: sweep one hyperparameter

**Setup.** Training and validation rows (or folds); a model with one knob (say ridge $\lambda$, or kNN's $k$, or tree depth).

**Held fixed.** All other hyperparameters, the preprocessing (fitted on training rows), the rows, the metric. Standardise before penalising, otherwise $\lambda$ penalises a feature in millimetres a thousand times more than the same feature in metres.

**What we manipulate.** The knob on a grid; for $\lambda$ a logarithmic grid $10^{-3}, 10^{-2}, \ldots, 10^{2}$, because the useful range spans orders of magnitude; for $k$ odd integers $1, 3, 5, \ldots$; for depth $1, 2, \ldots, 12$.

**What we measure.** Training and validation risk at every grid point.

**What the plot shows.** Training risk decreasing monotonically in capacity (so *increasing* in $\lambda$ and in $k$); validation risk U-shaped.

**How to read it.** The minimum of the validation curve is $\hat{h}$. A flat valley means the choice does not matter much; a sharp valley means it does. If the minimum is at the edge of the grid, extend the grid. Remember the direction of each knob: capacity grows with degree, depth, epochs, trees in boosting, and with *smaller* $\lambda$ and *smaller* $k$.

## Ridge on the four-point example

With $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$, the ridge slope is $\hat{w}_\lambda = 19/(14 + 4\lambda)$ [on your sheet]: 1.357 at $\lambda = 0$, 1.056 at $\lambda = 1$, 0.352 at $\lambda = 10$. Training error is smallest at $\lambda = 0$ by the nesting argument; whether $\lambda = 1$ is better for new points can only be seen on validation points. The same four points fitted by polynomials of degree 0, 1, 3 have training risks 1.25, 0.45, 0 and validation errors at the fifth point $(4, 5)$ of 6.25, 0.25, 100: the training column points to degree 3, the validation column to degree 1.

## Early stopping: the training curve over epochs

**Setup.** An iterative learner (gradient descent on a network, or boosting rounds), training and validation rows.

**Held fixed.** Everything but the number of iterations.

**What we manipulate.** The epoch $t$ (or boosting round $m$), by simply continuing to train and recording a checkpoint at each epoch.

**What we measure.** Training loss and validation loss after each epoch.

**What the plot shows.** Training loss falls throughout; validation loss falls, reaches a minimum at $t^*$, then rises.

**How to read it.** Keep the checkpoint $\hat{\theta}_{t^*}$ with the smallest validation loss [on your sheet], not the last one. A *patience* $p$ continues $p$ epochs past the current best before stopping, to avoid stopping on a wobble. The number of epochs is a capacity knob exactly like $\lambda$; stopping on the test loss is tuning on the test set.

## Coupled hyperparameters: learning rate, rounds and depth in boosting

**Setup.** Gradient-boosted trees, training and validation rows.

**Held fixed.** The data and the loss.

**What we manipulate.** Learning rate $\eta \in \{0.3, 0.1, 0.03\}$ and depth $\in \{2, 4, 6\}$ on a grid; for each pair, the number of rounds $M$ is chosen by early stopping on validation.

**What we measure.** The validation loss at the selected round, for each $(\eta, \text{depth})$.

**What the plot shows.** One validation curve over rounds per $(\eta, \text{depth})$. Halving $\eta$ moves the minimum to roughly twice as many rounds; deeper trees reach the minimum in fewer rounds.

**How to read it.** Never tune one of the three alone: the best $M$ depends on $\eta$ and depth. Compare configurations by their validation loss *at their own selected round*. Smaller $\eta$ usually ends a little lower at a higher cost.

## Tuning mistakes

- Choosing $\lambda$, $k$, depth or epochs on the training set (always maximal capacity, proof above), or on the test set (Section 3).
- A fine grid on a tiny validation set: the valley is noise; two configurations within one standard error are tied.
- Forgetting to standardise before an $L_2$ or $L_1$ penalty, or penalising the intercept.
- Reading the kNN curve backwards: capacity *increases* as $k$ decreases.
- Keeping the last epoch instead of the best checkpoint.
- After cross-validation, deploying a fold model instead of refitting $\hat{h}$ on all development rows.
- Tuning the decision threshold on training data or on test data; it is a hyperparameter and goes on validation.

## The knobs, their direction, and what they trade

| Knob | Capacity grows when | What is traded | Chosen on |
|---|---|---|---|
| polynomial degree, tree depth, network width | it increases | approximation error down, estimation error up | validation curve |
| $k$ in kNN | $k$ decreases | smoother average versus local sensitivity | validation curve |
| $\lambda$ (ridge, lasso) | $\lambda$ decreases | bias from shrinkage versus variance | log grid on validation |
| epochs / boosting rounds | they increase | fit of recurring patterns then of noise | early stopping on validation |
| number of bagged trees $M$ | does not grow capacity | stability only; OOB plateau | OOB or validation |
| $m_{\text{try}}$ in a forest | larger values make trees stronger and more alike | tree strength versus correlation | OOB or validation |

**Key insight.** A hyperparameter is any knob that training error would turn to maximal flexibility; so it is turned on data that did not fit the parameters, and its best value is read at the bottom of a U.

## How the professor asks it

**Q.** *Prove that the training error of ridge regression is minimised at $\lambda = 0$, and state what this implies for how $\lambda$ must be chosen.*
**A.** The penalised solution $\hat{\theta}_\lambda$ also solves the constrained problem $\min \widehat{R}_{\text{train}}$ s.t. $\Omega(\theta) \le c_\lambda$ with $c_\lambda = \Omega(\hat{\theta}_\lambda)$: if some feasible $\theta$ had smaller training risk, then $\widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c_\lambda < \widehat{R}(\hat{\theta}_\lambda) + \lambda c_\lambda$ would contradict optimality of $\hat{\theta}_\lambda$. The feasible sets shrink as $\lambda$ grows, and a minimum over a smaller set is at least as large, so $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in $\lambda$ and smallest at $\lambda = 0$. Therefore the training set would always choose no regularisation; $\lambda$ must be chosen on validation rows or by cross-validation, and the test set is kept for the final number.

**Q.** *Your boosting model is best at 400 rounds with learning rate 0.1. A colleague switches to*

*learning rate 0.01 and keeps 400 rounds, and reports worse validation loss. What happened and what should be done?*
**A.** With a ten times smaller step, 400 rounds apply about one tenth of the total correction, so the model is underfitted: the validation curve over rounds has not reached its minimum. Rounds, learning rate and depth are coupled; the number of rounds must be re-selected by early stopping for the new rate, which will land near 4,000 rounds, and the two configurations are then compared at their own selected rounds. The smaller rate may end slightly lower, at ten times the cost.

# 6 Learning curves: vary the amount of data

### Learning curve

A learning curve plots training and validation performance against the number of training rows $N$ used, with everything else fixed. [on your sheet]

**Purpose.** To answer "is more data worth collecting?" and to tell bias from variance: the plateau height is bias (plus noise), the gap is variance.

**Impact.** It is the only experiment that distinguishes "the family cannot express the relation" from "the family could, but the sample is too small": the first does not improve with $N$, the second does.

### The learning-curve protocol

**Setup.** Development rows and a fixed configuration $h$ (already tuned, or tuned at each size).

**Held fixed.** The configuration, the validation rows (or folds), the metric, the preprocessing rule.

**What we manipulate.** The number of training rows $N$: subsample $10\%, 20\%, \ldots, 100\%$ of the training set (nested subsamples, so that each larger set contains the smaller).

**What we measure.** At each $N$: training risk of the fitted model on its own $N$ rows, and validation risk on the fixed validation rows. Repeat with several random subsamples at each $N$ and average.

**What the plot shows.** Training risk *rises* with $N$ (more rows are harder to fit perfectly); validation risk *falls* with $N$; the two approach each other.

**How to read it.** Three shapes. (1) Both curves high and already flat, close together: bias; more data will not help; change the representation or increase capacity. (2) Training low, validation high, wide gap that narrows slowly: variance; regularise, reduce capacity, or collect more representative data (the gap will close with $N$). (3) Both still falling and converging: data-limited; more rows are likely to help. A validation curve that *worsens* over calendar time is a fourth case, distribution shift, not visible on a learning curve.

### Why the two curves bracket the same limit

Fix the family $\mathcal{H}$ with best member $f^*_{\mathcal{H}}$. For the fitted model on $N$ rows, the sandwich of Section 2 holds at every $N$:

$$\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \;\le\; R(f^*_{\mathcal{H}}) \;\le\; \mathbb{E}[R(\hat{f}_N)].$$

As $N$ grows, the empirical risk of every member of a reasonable family approaches its population risk uniformly, so the fitted model approaches $f^*_{\mathcal{H}}$ and both outer terms approach $R(f^*_{\mathcal{H}})$: the training curve rises to it from below, the validation curve falls to it from above. The common limit is the *approximation error* of the family plus the irreducible noise; it does not depend on $N$ and cannot be bought with data. The gap between the curves at finite $N$ is the *estimation error*; it shrinks with $N$ and grows with capacity. In the bias–variance language at a fixed $x$, $\mathbb{E}(Y - \hat{f}(x))^2 = \sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]: the plateau is $\sigma^2 + \text{Bias}^2$, the gap is mostly Var.

**Should the security team label 20,000 more flows?**

The team has 10,000 labelled flows. They train a random forest on 1,000, 2,000, ..., 8,000 and validate on the same 2,000. Validation recall goes $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ while training recall stays near 1.0: a large gap that is closing slowly and a validation curve that has almost flattened. Doubling the data would move recall from 0.88 to perhaps 0.89; the remaining gap is variance that regularisation (larger leaves, fewer features per split) addresses more cheaply, and the plateau says the features carry no more information. Labelling 20,000 more flows is not the best use of the budget. Had the validation curve still been rising steeply at 8,000, the answer would be yes.

**Learning-curve mistakes**

- Confusing the three horizontal axes: capacity (validation curve), epochs (training curve), data size (learning curve). Each has its own shape and its own diagnosis.
- Re-tuning nothing when $N$ changes: the best $\lambda$ or depth depends on $N$; at small $N$ a tuned model is more regularised. Either tune at each size or state that the configuration was fixed.
- Using non-nested random subsamples, so that noise between subsamples hides the trend; or a single run per size.
- Believing more data removes bias. It removes variance; the plateau is set by the family and the features.
- Reading "errors close and low" as "done": close and low on a random split can still be a shortcut; the protocol question of Section 2 comes first.

**Reading the curves: the diagnostic table**

| Observed pattern | Likely issue | Response |
|---|---|---|
| training and validation both high, flat in $N$ | underfitting (bias) | richer representation, more capacity, less regularisation |
| training low, validation high, gap narrows slowly with $N$ | high variance | regularise, reduce capacity, more representative data, ensembles |
| both decreasing and converging | data-limited | more representative examples will help |
| validation fine on the split, degrades over calendar time | distribution shift | temporal evaluation, retraining schedule, monitoring |

[on your sheet]

**Key insight.** Data buys variance reduction, not bias reduction: the learning curve's gap closes with $N$, its plateau does not move.

**How the professor asks it**

**Q.** *A learning curve shows training error at 2% and validation error at 15% at every data size from 1,000 to 50,000 rows, with no narrowing. Which diagnosis does the table suggest, and why might the table be wrong here?*

**A.** The table says high variance (large gap). But variance shrinks with $N$, and a gap that does not narrow across a fiftyfold increase in data is not sampling variance: it is a systematic difference between the training rows and the validation rows. Typical causes are a shortcut present in the training collection and absent from the validation protocol (a grouped or temporal validation set), duplicates in the training rows, or label noise concentrated in one subset. The response is

not more data or regularisation but an audit of the split and the features.

**Q.** *Explain, using the sandwich inequality, why the training curve of a learning curve rises with $N$ while the validation curve falls.*
**A.** At every $N$, $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_N)]$. With few rows the minimiser can fit the sample's accidents, so the training risk is far below $R(f^*_{\mathcal{H}})$ and the true risk far above it; with many rows the empirical risk of every member of the family is close to its population risk, so the fitted model is close to $f^*_{\mathcal{H}}$ and both terms approach $R(f^*_{\mathcal{H}})$. The lower term therefore rises toward the limit and the upper term falls toward it. The limit is the approximation error plus noise, which $N$ cannot change.

# 7 Bias–variance experiments

## Bias and variance of a procedure

At a fixed input $x$, with $f^*(x) = \mathbb{E}[Y \mid X = x]$ and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average prediction over training sets,

$$\text{Bias}(x) = \bar{f}(x) - f^*(x), \qquad \text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2], \quad \text{[on your sheet]}$$

and $\mathbb{E}_{\mathcal{D},Y|x}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}(x)^2 + \text{Var}(x)$ [on your sheet]. These are properties of the *procedure* (family plus algorithm plus $N$), not of one fitted model: one model has no variance.

**Purpose.** To split the error at $x$ into the part the family cannot fix (bias), the part the sample causes (variance), and the part nothing can fix (noise).

**Impact.** Capacity moves bias down and variance up; data moves variance down; regularisation and averaging trade a little bias for a lot of variance.

## The decomposition, line by line

Write $Y = f^*(x) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid x] = 0$, $\text{Var}(\varepsilon \mid x) = \sigma^2(x)$, and the training set independent of the test pair.
*Step 1, split off the noise.* $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^*(x) - \hat{f}_{\mathcal{D}}(x))$. Square: $\varepsilon^2 + 2\varepsilon(f^* - \hat{f}_{\mathcal{D}}) + (f^* - \hat{f}_{\mathcal{D}})^2$. Expectation: $\mathbb{E}[\varepsilon^2] = \sigma^2$; the cross term is $\mathbb{E}[\varepsilon] \cdot \mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}] = 0$ because $\varepsilon$ is independent of $\mathcal{D}$ and has mean zero.
*Step 2, split around the average prediction.* $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$. The first bracket is the constant $-\text{Bias}(x)$; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by definition of $\bar{f}$. Square and take $\mathbb{E}_{\mathcal{D}}$: $\text{Bias}^2 + 0 + \text{Var}$.
Both steps are the same move: add and subtract a mean, square, and let the cross term die because a mean-zero quantity multiplies a constant.

## Simulating bias and variance

**Setup.** A world you control: $f^*(x) = \sin(2\pi x)$, noise $\sigma = 0.3$, $N = 25$ training points per set, a grid of 50 test inputs.

**Held fixed.** The truth, the noise, $N$, the test grid.

**What we manipulate.** Two things, in two separate runs: (a) the training set, drawn 500 times (this is the repetition that defines $\mathbb{E}_{\mathcal{D}}$); (b) the capacity: polynomial degree $0, 1, 3, 5, 9$, or $k$ in kNN.

**What we measure.** For each capacity and each test input: the 500 predictions; their mean $\bar{f}(x)$; $\text{Bias}^2(x) = (\bar{f}(x) - f^*(x))^2$; $\text{Var}(x)$ as their spread; average both over the grid.

**What the plot shows.** $\text{Bias}^2$ falling with capacity, variance rising, their sum plus $\sigma^2$ U-shaped

and matching the measured test error.

**How to read it.** The U of the validation curve is the sum of a falling and a rising term. Degree 0 is all bias; degree 9 on 25 points is all variance. Now repeat run (b) with $N = 250$: the variance curve drops everywhere, the bias curve does not move, and the minimum of the U shifts to a higher degree. That is the whole story of "more data allows a bigger model".

## Bias and variance computed exactly: estimate a mean

No input; $Y = \mu + \varepsilon$, $\mathrm{Var}(\varepsilon) = \sigma^2 = 1$, $N = 4$ draws. Three procedures. *Sample mean*: unbiased, variance $\sigma^2/N = 0.25$. *Ignore the data, predict 0*: bias $-\mu$, variance 0. *Shrunk mean*, half the sample mean: bias $-\mu/2$, variance $0.25/4 = 0.0625$. Bias$^2$+variance: for $\mu = 1$: 0.25, 1, 0.3125; for $\mu = 0.2$: 0.25, 0.04, 0.0725. When the truth is far from the shrinkage target the unbiased procedure wins; when it is near, the biased ones win by trading a little bias for a large variance reduction. Ridge regression is this trade with a dial: $\hat{w}_\lambda$ is the least-squares slope times a factor in $(0, 1]$.

## Variance reduction by averaging: the ensemble-size curve

**Setup.** Bootstrap samples of one training set, a deep tree per sample, a validation set.

**Held fixed.** The data, the tree settings.

**What we manipulate.** The number of trees $M$: $1, 5, 10, 25, 50, 100, 200$; and, in a second run, $m_{\text{try}}$ (all features versus $\sqrt{d}$).

**What we measure.** Validation error of the average of the first $M$ trees; optionally, the variance of the ensemble prediction at a few fixed inputs across bootstrap replicates.

**What the plot shows.** Error falling steeply then flattening. The curve with $m_{\text{try}} = \sqrt{d}$ flattens *lower*.

**How to read it.** The floor is $\rho\sigma^2$ in $\mathrm{Var}(\bar{e}) = \sigma^2(\rho + (1-\rho)/M)$ [on your sheet]: with $\rho = 0.2$ and $M = 25$, $0.232\sigma^2$, and never below $0.2\sigma^2$ however many trees. Feature randomisation lowers $\rho$, which lowers the floor; that is why the forest beats bagging. Averaging does nothing for bias: if every tree lacks the same feature, every error points the same way and $\rho \approx 1$.

## Bias–variance mistakes

- Speaking of "the variance of this model". Variance is over training sets; one fitted model is one draw. The experiment that measures it must refit many times.
- Expecting more data to reduce bias, or regularisation to reduce variance for free: regularisation adds bias (shrinkage) and removes variance; whether the sum falls depends on where the truth is (the $\mu = 1$ versus $\mu = 0.2$ table).
- Treating "low bias, low variance at moderate capacity" as achievable in general: at the bottom of the U both terms are positive, just small.
- Forgetting that the decomposition is for squared error at a fixed $x$ with training noise independent of test noise; for classification the "bias" and "variance" of a 0–1 loss do not add this way.
- Assuming the U always holds: very large models can show double descent, with test error peaking near the interpolation threshold (parameters $\approx N$) and falling again beyond [on your sheet]. The classical U is the first model, not a law.

**What each lever does to bias and variance**

| Lever | Bias | Variance | Note |
|---|---|---|---|
| more capacity | down | up | U-shaped sum |
| more data $N$ | unchanged | down | plateau unchanged |
| regularisation $\lambda$, shrinkage | up | down | worth it when the truth is near the target |
| early stopping | up slightly | down | capacity through training time |
| bagging, forests | unchanged | down to the floor $\rho\sigma^2$ | shared errors survive |
| boosting | down | up with rounds | early stopping on rounds |
| better features | down | often down | attacks the plateau |

**Key insight.** Bias is what repeating the experiment never averages away; variance is what a different sample would change; and the experiment that separates them is "refit many times from fresh samples and look at the cloud of predictions at one $x$".

**How the professor asks it**

**Q.** *A 1-nearest-neighbour regressor and a 50-nearest-neighbour regressor are both fitted on the same 200 points. Describe the bias–variance experiment that compares them and predict its outcome.*
**A.** Draw many training sets of 200 points from the same world, fit both regressors on each, record both predictions at a fixed $x$, and compute the spread of each cloud (variance) and the distance of each cloud's centre from $f^*(x)$ (bias). The 1-NN prediction is one noisy neighbour's $y$, so its cloud is as wide as the noise, Var $\approx \sigma^2$, and its centre is close to $f^*(x)$: low bias, high variance. The 50-NN prediction averages 50 neighbours, so its spread is about $\sigma^2/50$, but the neighbours reach far from $x$ and their average follows a smoothed version of $f^*$: higher bias, low variance. On the sheet: Bias $= \frac{1}{k}\sum_{j\le k} f(x^{(j)}) - f(x)$, Var $= \sigma^2/k$ [on your sheet]. Which total is smaller depends on how curved $f^*$ is relative to the noise; the validation curve over $k$ finds the balance.

**Q.** *Give a counterexample to "averaging many models always reduces error".*
**A.** Take $M$ copies of the same fitted model, or $M$ models that all lack the same essential feature. Their errors at any $x$ are identical ($\rho = 1$), so $\mathrm{Var}(\bar{e}) = \sigma^2(1+0) = \sigma^2$: the average has the error of one model. More generally the floor $\rho\sigma^2$ is untouched by $M$; averaging removes only the part of the error that differs between models, never shared bias.

# 8 The complete protocol, start to finish

**The experiment everyone runs, as twelve steps**

1. **State the deployment question** (new cases of what kind: future? new devices? same population?). This decides the split rule.
2. **Audit the data**: duplicates, conflicting labels, missing values, class imbalance, features known only after the outcome.
3. **Freeze the test set** according to the split rule (random, stratified, grouped, temporal). Write down its row identifiers. Do not look at it again.
4. **Fix the metric from the costs**: accuracy hides minority failure; use recall, precision, specificity, balanced accuracy, $F_1$ [on your sheet] as the costs dictate; on 10,000 flows with 50 attacks, TP = 40, FN = 10, FP = 90 gives accuracy 0.99 and precision 0.31.
5. **Fit baselines** (majority class, a shallow tree, logistic regression, kNN) under the same protocol; anything you build must beat them.

6. **Build the pipeline** so that every data-dependent step (scaler, imputer, selector, model) is fitted on training rows only.
7. **Tune** each hyperparameter on validation rows or by cross-validation on a log or integer grid; read the U; treat near-ties as ties; choose the number of epochs or rounds by early stopping; choose the decision threshold on validation.
8. **Diagnose** with learning curves (data-limited? bias? variance?) and, where instability is suspected, with repeated fits over seeds or bootstrap samples.
9. **Probe shortcuts**: shuffle the labels (performance must collapse to chance), drop metadata columns (performance must not collapse), evaluate under the deployment-like protocol.
10. **Refit the chosen configuration on all development rows** (training plus validation).
11. **Evaluate once on the test set**; report the number with its uncertainty; do not go back.
12. **Monitor in deployment**: a degrading metric over calendar time is distribution shift; schedule retraining and keep a time-aware evaluation.

## The ten mistakes of this document

1. Grading a model on the rows that chose it (training rows for $\theta$, validation rows for $h$).
2. Fitting the scaler, imputer or feature selector on all rows before splitting, or outside the cross-validation folds.
3. A random split when deployment is in the future (temporal) or on new entities (grouped).
4. Looking at the test set more than once, including early stopping or threshold choice on it.
5. Reporting the winner's validation or cross-validation score as the final estimate.
6. Choosing $\lambda$, $k$, depth or epochs on training error (always maximal capacity).
7. Confusing the three curves: capacity axis, epoch axis, data-size axis.
8. Expecting more data to remove bias, or an ensemble to remove shared error.
9. Tuning boosting's rate, rounds and depth separately.
10. Comparing candidates on different fold assignments, on a tiny validation set, or deploying a fold model instead of refitting.

## The experiments in ten lines

1. Every experiment: hold all fixed, vary one knob, measure risk on fitting rows and on unseen rows, read two curves.
2. Fixed model: held-out risk is honest. Fitted model: training risk is optimistic, $\mathbb{E}\widehat{R}_{\text{train}}(\hat{f}) \le R(f^*_{\mathcal{H}}) \le \mathbb{E}R(\hat{f})$.
3. Three sets, three jobs: train fits $\theta$, validation chooses $h$, test reports once. The split rule comes from the deployment question.
4. Leakage is tomorrow's information today: preprocessing on all rows, duplicates, groups across sets, post-outcome features, shuffled time.
5. Winner's curse: the minimum of honest estimates is optimistic; it grows with noise $\sigma$ linearly and with the number of candidates like $\sqrt{2\ln n}$.
6. Cross-validation holds out each fold in turn; the whole pipeline is refitted inside each fold; refit on all development rows afterwards; nested CV when you need an honest number without a test set.
7. Hyperparameters are knobs training error would turn to maximal capacity; choose them at the bottom of the validation U; epochs by early stopping; knobs in boosting are coupled.
8. Learning curves: training rises, validation falls, both converge to approximation error plus noise; gap is variance, plateau is bias.
9. Bias–variance: refit on many samples, look at the cloud at one $x$; squared error $= \sigma^2 + \text{Bias}^2 + \text{Var}$; capacity trades them, data lowers variance, averaging lowers variance to the floor $\rho\sigma^2$.
10. The protocol: question, audit, freeze test, metric from costs, baselines, pipeline inside folds, tune on validation, diagnose with curves, probe shortcuts, refit, test once, monitor.